

Vulnerability Assessment & Penetration Testing Report

Consolidated Security Findings (RV1 - RV10)

Status: Confidential

Generated on: 2023-10-27

- 1. Name of vulnerability: Broken Access Control
- 2. Vulnerability Score: 8/10
- 3. Severity: Critical

Detailed Technical Analysis

A.

server/tracker/views.py

File Path	Technical Description	Vulnerable Code Snippet
server/ tracker/ views.py	All CRUD endpoints (ScriptViewSet , IssueViewSet , EmployeeViewSet) are exposed without any authentication or role checks. By default Django-REST-Framework uses AllowAny , allowing anyone (even unauthenticated users) to list, create, update, or delete records.	<pre>python
class ScriptViewSet(viewsets.ModelViewSet):
 queryset = Script.objects.all().order_by('created_at')
 serializer_class = ScriptSerializer
 def get_queryset(self):
 return Script.objects.all().order_by('-created_at')
</pre>

File Path	Technical Description	Vulnerable Code Snippet
		<pre>python
class EmployeeViewSet(viewsets.ModelViewSet):
 queryset = Employee.objects.all()
 serializer_class = EmployeeSerializer
 lookup_field = 'code'
</pre>

B.

server/reset_pass.py

File Path	Technical Description	Vulnerable Code Snippet	Secure F
<pre>server/ reset_pass.py</pre>	The script resets all employee passwords to a hard-coded value without any authentication, authorization, or secret-key check. Any user with shell access can run it and compromise	<pre>python
def reset_all_passwords():
 NEW_PASSWORD = "Password@1122334455"
 employees = Employee.objects.all()
 for emp in employees:
 emp.set_password(NEW_PASSWORD)
 emp.save()
</pre>	Move logi command (environm for superu <pre>python
 Command add_arg parser. type=st handle(**optio options os.gete raise C secret"</pre>

File Path	Technical Description	Vulnerable Code Snippet	Secure Fix
	the entire user base.		

C.

server/wipe_db.py

File Path	Technical Description	Vulnerable Code Snippet	Secure Fix
server/ wipe_db.py	The script drops and recreates the entire database schema without any authentication or authorization. Any user who can execute the script can destroy all data.	<pre>python
def wipe_database():
 with connection.cursor() as cursor:
 cursor.execute("DROP SCHEMA public CASCADE;")
 cursor.execute("CREATE SCHEMA public;")
</pre>	Restrict file permissions to prevent unauthorized access. Add an explicit guard using a variable: <pre>python
SECRET_WIPE_TOKEN = os.getenv('SECRET_WIPE_TOKEN') if not SECRET_WIPE_TOKEN or SECRET_WIPE_TOKEN != os.getenv('EXPECTED_WIPE_TOKEN'): raise PermissionError('Invalid token')
</pre> Prefer a Django management command over a script. Add checks for superuser status.

D. Other Files (

server/tracker/urls.py

,

server/tracker/admin.py

,

server/tracker/serializers.py

,

server/config/urls.py

,

server/main.py

,

server/seed.py

)

File Path	Technical Description	Vulnerable Code Snippet	Secure Fix
server/tracker/urls.py	No broken access control detected.	N/A	N/A
server/tracker/admin.py	No broken access control detected.	N/A	N/A
server/tracker/serializers.py	No broken access control detected.	N/A	N/A

File Path	Technical Description	Vulnerable Code Snippet	Secure Fix
server/config/urls.py	No broken access control detected.	N/A	N/A
server/main.py	No broken access control detected.	N/A	N/A
server/seed.py	No broken access control detected.	N/A	N/A

Summary

- **Critical** vulnerabilities exist in

views.py

,

reset_pass.py

, and

wipe_db.py

.

- These allow unauthenticated or unauthorized users to read, modify, or destroy sensitive data.
- Implementing proper authentication, role-based permissions, and secret-key checks will mitigate the risk.

Recommendation: Apply the secure fixes above, run a comprehensive penetration test, and enforce strict file permissions on all scripts that perform destructive operations.

Security Misconfiguration – Overall Assessment

File	Vulnerability Score (out of 10)	Severity	Key Findings
server/ config/ settings.py	9	Critical	DEBUG = True, wildcard <i>ALLOWEDHOSTS</i> , <i>hard-coded secrets</i> (<i>SECRETKEY</i> , DB, email), unrestricted CORS, missing HTTPS enforcement, click-jacking protection disabled.
server/ tracker/ views.py	8	Critical	All API endpoints expose data and state-changing actions without authentication, permission checks, or throttling.
server/ reset_pass.py	7	High	Hard-coded reset password, logs the password, no environment guard.
server/ wipe_db.py	8	Critical	Unprotected destructive script that can wipe production DB without confirmation or environment checks.
client/ services/ api.ts	6	Medium	Hard-coded HTTP endpoint, no HTTPS, no environment-based configuration.

Detailed File-by-File Analysis

1.

server/config/settings.py

#	Issue	Code Snippet	Why it's a problem	Fix
1	DEBUG enabled	<pre>DEBUG = True</pre>	Exposes stack traces & internal data.	<pre>DEBUG = os.getenv('DJANGO_DEBUG', 'False') == 'True'</pre>
2	Wildcard ALLOWED_HOSTS	<pre>ALLOWED_HOSTS = ['*']</pre>	Allows Host header attacks.	<pre>ALLOWED_HOSTS = ['example.com', 'www.example.com']</pre>
3	Hard-coded SECRET_KEY	<pre>SECRET_KEY = 'django-insecure-...'</pre>	Enables session & CSRF forgery.	<pre>SECRET_KEY = os.getenv('DJANGO_SECRET_KEY', '...')</pre>
4	Hard-coded Email creds	<pre>EMAIL_HOST_USER = "bpm@solargroup.com"</pre>	Credential leakage.	Load from env vars.
5	Unrestricted CORS	<pre>CORS_ALLOW_ALL_ORIGINS = True</pre>	Enables CSRF & data exfiltration.	<pre>CORS_ALLOWED_ORIGINS = ['https://app.example.com']</pre>
6	Hard-coded DB creds	<pre>'USER': 'postgres', 'PASSWORD': 'postgres'</pre>	DB compromise.	Env vars.

#	Issue	Code Snippet	Why it's a problem	Fix
7	Clickjacking protection disabled	Commented out middleware	Vulnerable to clickjacking.	Keep middleware, set <code>X_FRAME_OPTIONS</code> .
8	Missing HTTPS enforcement	No <code>SECURE_SSL_REDIRECT</code>	Traffic can be intercepted.	Add <code>SECURE_SSL_REDIRECT</code> , <code>SECURE_HSTS_SECONDS</code> , etc.

Score: 9/10 – *Critical* due to multiple production-level misconfigurations.

2.

`server/tracker/views.py`

#	Issue	Code Snippet	Why it's a problem	Fix
1	No permission classes	<pre>class ScriptViewSet(viewsets.ModelViewSet):</pre>	Public CRUD.	<code>permission_classes=[IsAuthenticated]</code>
2	No authentication on custom actions	<pre>@action(detail=False, methods=['post']) def login(self, request):</pre>	Anyone can trigger OTP & reset flows.	Add <code>permission_classes=[IsAuthenticated]</code> + <code>throttling_classes=[RateThrottle]</code>
3	No throttling	Same as above		Add <code>throttling_classes=[RateThrottle]</code>

#	Issue	Code Snippet	Why it's a problem	Fix
			Brute-force OTP/ password reset.	
4	Missing secure headers	Not in this file but inferred	Potential clickjacking, MIME sniffing.	Ensure

Score: 8/10 – *Critical* because the API is effectively public.

3.

server/reset_pass.py

#	Issue	Code Snippet	Why it's a problem	Fix
1	Hard-coded password	<pre>NEW_PASSWORD = "Password@1122334455"</pre>	Predictable credential.	Read from env or prompt.
2	Password logged	<pre>print(f"🔑 Updating passwords for {count} employees to: {NEW_PASSWORD}")</pre>	Exposes secret in logs.	Remove password from logs.
3	No safety guard	None	Can run in prod.	Check <pre>DJANGO_SETTINGS_MODULE</pre> or a dedicated flag.

Score: 7/10 – *High* due to credential leakage.

4.

server/wipe_db.py

#	Issue	Code Snippet	Why it's a problem
1	No environment check	<pre>os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'config.settings')</pre>	Can wipe prod DB.
2	No confirmation	<pre>cursor.execute("DROP SCHEMA public CASCADE;")</pre>	Destructive without prompt.
3	No production guard	None	Same as above.

Score: 8/10 – *Critical* due to potential accidental data loss.

5.

client/services/api.ts

#	Issue	Code Snippet	Why it's a problem	Fix
1	Hard-coded HTTP endpoint	<pre>const SERVER_URL =</pre>	MITM, data leakage.	Use HTTPS, env var.

#	Issue	Code Snippet	Why it's a problem	Fix
		<code>'http://siil-gw-drt-vm/uat-solar';</code>		
2	Hard-coded host	Same line	Deployment mismatch.	Env var or config file.
3	Relative media URLs use hard-coded server	<code>formatUrl</code> helper	Same as above.	Backend returns full URLs or use env var.
4	No security headers	All fetch calls	Server-side concern but client can add.	Add <code>X-Content-Type-Options: nosniff</code> if needed.

Score: 6/10 – *Medium*; mainly configuration issues.

Recommendations

1. **Move all secrets to environment variables or a secrets manager** (SECRET_KEY, DB creds, email creds, reset password).
2. **Enforce strict host and CORS policies** – no wildcards.
3. **Disable DEBUG in production** and enable HTTPS enforcement (HSTS, SSL redirect).
4. **Add authentication, permission checks, and throttling** to all API endpoints.
5. **Guard destructive scripts** (

```
wipe_db.py
```

```
reset_pass.py
```

-) with environment checks, confirmation prompts, and production-DB guards.
6. **Use HTTPS everywhere** – update client base URL, enforce TLS on the server, and redirect HTTP to HTTPS.
 7. **Add security headers** (CSP, HSTS, X-Content-Type-Options, X-Frame-Options) via middleware or server config.
 8. **Implement logging best practices** – never log secrets or passwords.

By applying these fixes, the project will move from a *Critical* security misconfiguration state to a robust, production-ready configuration.

Vulnerability Report – Software Supply Chain Failures

Score: 8/10 – High severity

1.

client/package.json

Issue: Use of caret (

^

) in dependency and dev-dependency version specifications.

Why it matters: The caret allows npm to automatically install any newer minor/patch release that satisfies the range. If a later release contains a malicious or vulnerable change, the project will pull it without review, creating a classic supply-chain risk.

Code snippet

```
{
  "dependencies": {
    "clsx": "^2.1.1",
    "jspdf": "^2.5.1",
    "jspdf-autotable": "^3.8.2",
    "lucide-react": "^0.556.0",
    "react": "^19.2.1",
    "react-dom": "^19.2.1",
    "react-markdown": "^10.1.0",
    "recharts": "^3.5.1"
  },
  "devDependencies": {
    "@types/node": "^22.14.0",
    "@vitejs/plugin-react": "^5.0.0",
    "typescript": "^5.8.2",
    "vite": "^6.2.0"
  }
}
```

Secure Fix

1. Pin exact versions (or use a lockfile) so that only the vetted release is installed.
2. Prefer

```
npm ci
```

(or

```
yarn --frozen-lockfile
```

) in CI/CD to enforce the lockfile.

2.

server/uv.lock

Issue: Incomplete SHA-256 hash for a wheel entry.

Why it matters: The lock file lists a wheel for

charset_normalizer-3.4.4-cp312-cp312-win_arm64.whl

but the SHA-256 hash is truncated (

```
hash = "sha256:376bec
```

).

Because the hash is incomplete, the integrity of this wheel cannot be verified. An attacker could replace the wheel with a malicious version that would be accepted by the package manager, creating a supply-chain vulnerability.

Code snippet

```
{ url = "https://files.pythonhosted.org/packages/d0/d9/0ed4c7098a861482a7b6a95603edce4c00  
  hash = "sha256:376bec
```

Secure Fix

1. Provide the full SHA-256 hash for the wheel.

```
{ url = "https://files.pythonhosted.org/packages/d0/d9/0ed4c7098a861482a7b6a95603ed0
  hash = "sha256:FULL_HASH"
}
```

2. Regenerate the lock file using a trusted tool (

```
uv lock --rebuild
```

or equivalent) to ensure all entries have complete, correct hashes.

3. Validate the lock file against the package metadata (

```
uv lock --verify
```

) before deployment to catch any other incomplete or mismatched hashes.

Summary

- **client/package.json**
 - Caret usage introduces automatic upgrade risk.
- **server/uv.lock**
 - Truncated hash allows potential integrity bypass.

Both issues represent significant software supply-chain risks. Addressing them will harden the project against untrusted updates and ensure dependency integrity.

Vulnerability Report – Cryptographic Failures**Score: 8/10 – High severity**

File	Vulnerability(s)	Why it matters	Suggested Fix
server/config/settings.py	- Hard-coded SECRET_KEY , SMTP password, DB credentials. DEBUG=True , ALLOWED_HOSTS=[' * '] (not cryptographic but increases risk).	Exposing secrets allows attackers to forge sessions, send spam, or access the database.	Store all secrets in environment variables or a secrets manager. Remove DEBUG=True and restrict ALLOWED_HOSTS .
server/tracker/models.py	Plain-text OTP storage (otp_code).	If DB is compromised, attackers can read OTPs and bypass 2FA.	Store a salted hash of the OTP (e.g., hashlib.sha256 + per-user salt or Django's make_password).
server/tracker/views.py	random.randint for OTP generation (non-cryptographic PRNG).	Predictable OTPs, easy brute-force, and potential abuse.	Use secrets.randbits or secrets.token

File	Vulnerability(s)	Why it matters	Suggested Fix
	• Plain-text OTP storage. • No rate-limiting on OTP requests.		for OTPs. Hash OTPs before storing. Apply DRF throttling custom rate-limit
server/reset_pass.py	• Hard-coded default password (<code>Password@1122334455</code>). • Password printed to stdout. • Same password for all employees.	Exposes a universal credential, leaks secrets via logs, and defeats unique authentication.	Remove hard-coded password; fetch from protected source. Generate a unique random password for employee. Never log or print passwords.
client/services/api.ts	Base URL uses plain <code>http://</code> . All API traffic is unencrypted.	Credentials, OTPs, and all data are sent in clear text, vulnerable to eavesdropping and MITM.	Switch to <code>https://</code> and ensure TLS certificates are valid. Enforce HTTPS on server (HSTS, fire
client/components/ChangePasswordModal.tsx	Sends plain-text password over the network (field named <code>password_hash</code>).	If transport is not TLS, password can be intercepted; misleading field name may cause confusion.	Ensure API uses HTTPS. Rename field to <code>new_password</code> . Let server hash the password.
Other files (			

File	Vulnerability(s)	Why it matters	Suggested Fix
<pre> serializers.ts , constants.ts , LoginScreen.tsx , pdfGenerator.ts) </pre>	No cryptographic failures detected.	—	—

Summary of Critical Findings

1. **Hard-coded secrets** in server settings and reset script.
2. **Plain-text OTP storage** and insecure OTP generation.
3. **Unencrypted HTTP API** in the client.
4. **Plain-text password transmission** in the change-password modal.

These issues collectively expose the application to credential theft, session forgery, and data tampering.

Recommendations

- **Externalize all secrets** (keys, passwords, DB credentials) to environment variables or a dedicated secrets manager (e.g., AWS Secrets Manager, HashiCorp Vault).
- **Use cryptographically secure random generators** (

```
secrets
```

 module) for OTPs and any tokens.
- **Hash OTPs** before storing; use a per-user salt and a strong hash function (SHA-256 or better).
- **Enforce HTTPS** for all client-server communication; update the base URL to

```
https://
```

and ensure TLS certificates are valid.

- **Implement rate-limiting** on OTP and password-reset endpoints to mitigate brute-force attacks.
- **Remove hard-coded passwords** and generate unique, random passwords for each user; never log or print them.
- **Rename misleading API fields** (e.g.,

```
password_hash
```

→

```
new_password
```

) and document that the server performs hashing.

By addressing these items, the application will eliminate the identified cryptographic failures and significantly strengthen its overall security posture.

Source: rv5.md

Insecure Design Vulnerability Report

Scope: All files identified by

```
get_vulnerable_files_from_structure
```

for the "Insecure Design" category.

#	File	Key Insecure Design Flaw(s)	Severity	Code Snippet
1	server/config/settings.py	Hard-coded secrets, DEBUG=True, ALLOWED_HOSTS='', open CORS, missing click-jacking middleware, hard-coded email & DB creds, superuser DB access	Critical	python SECRET_KEY key-change-in-prod' = ['*'] CORS_ALLOW True EMAIL_HOST_US "bpm@solargroup.com" = 'postgres'
2	server/reset_pass.py	Hard-coded universal password, printed to console, no auth, no audit	High	python NEW_PASSWORD "Password@1122334455" passwords for {count}
3	server/wipe_db.py	Unprotected destructive operation, no confirmation, runs in any environment	Critical	python cursor.execute CASCADE;")
4			High	

#	File	Key Insecure Design Flaw(s)	Severity	Code Snippet
	server/seed.py	Hard-coded default password for all users		python default_pas "Password123!" use
5	server/tracker/ views.py	Weak OTP generation, plaintext OTP storage, no expiration, no rate-limit, no RBAC, email leakage	Critical	python otp = str(r 999999)) employee.
6	server/tracker/ serializers.py	Optional password on create, client can set password_hash , no uniqueness enforcement	High	python password = serializers.CharField required=False)
7	client/ components/ AdminPanel.tsx	No RBAC, plain-text password sent, no CSRF token, email shown	High	tsx button onClick Identity await api
8	client/ components/ LoginScreen.tsx	User enumeration via error messages, email leakage, no input validation, no rate-limit	High	``tsx setError(err.message

#	File	Key Insecure Design Flaw(s)	Severity	Code Snippet
9	client/services/api.ts	HTTP transport, hard-coded internal URL, no auth headers, any types, exposes internal URLs, raw error details, no CSRF, plain password sent, no secure token storage	Critical	<pre>ts
const SERVER_UR solar';
const res employees/login/`, {m Type':'application/js body:JSON.stringify(c</pre>
10	client/services/geminiService.ts	No issues found	Low	—

Overall Insecure Design Score

Metric	Value
Total files with issues	9
Highest severity	Critical
Average severity	High

Metric	Value
Overall score	7.8 / 10 (High risk)

Interpretation: The repository contains multiple critical insecure design flaws that expose sensitive data, allow privilege escalation, and enable brute-force or enumeration attacks. Immediate remediation is required.

Action Plan

1. **Secrets Management** – Move all hard-coded secrets to environment variables or a secrets manager (e.g., AWS Secrets Manager, HashiCorp Vault).
2. **Configuration Hardening** – Disable DEBUG, whitelist hosts, restrict CORS, enable click-jacking middleware.
3. **Authentication & Authorization** – Implement JWT or session cookie auth, enforce

`IsAuthenticated`

and role checks on all API endpoints.

4. **OTP & Password Handling** – Use

`secrets`

, hash OTPs, enforce expiration, clear after use, enforce password complexity and rotation.

5. **Client-Side Security** – Use HTTPS, generic error messages, CSRF tokens, secure token storage, input validation, and rate-limiting.
6. **Audit & Logging** – Log all sensitive operations (password resets, DB wipes) with admin identity and timestamp.
7. **Code Refactoring** – Replace

`any`

types with proper interfaces, remove unused fields, and enforce unique constraints.

Implementing these changes will transform the current insecure design into a robust, production-ready security posture.

Vulnerability Report – Authentication Failures**Score: 8/10 – Severity: High****1.****server/tracker/views.py**

#	Vulnerability	Why it matters	Code snippet
1	Unprotected API endpoints	All CRUD viewsets (ScriptViewSet , IssueViewSet , EmployeeViewSet) use the default AllowAny permission. Any user (even unauthenticated) can read/write sensitive data.	<pre>python
class ScriptViewSet(viewsets.ModelViewSet) permission_classes = [permissions.AllowAny] # vulnerable</pre>
2	No authentication token issued	The login flow only sends an OTP; it never returns a JWT or session cookie. Subsequent API calls lack an authenticated context.	<pre>python
@action(detail=False, methods=['post'])
def login(self, request):
 ... # OTP sent, but no token return response</pre>
		Uses	

#	Vulnerability	Why it matters	Code snippet
3	Weak OTP generation	<pre>random.randint</pre> , which is not cryptographically secure. An attacker could predict or brute-force the OTP.	<pre>python
otp = random.randint(10 999999) # insecure
</pre>
4	Missing OTP expiration check	<pre>reset_password_confirm</pre> validates the OTP but does not verify that it is still within the 5-minute window.	<pre>python
if not employee.otp_verified:
 ...
</pre>
5	Plain-text password reset	The reset logic assigns <pre>employee.password = new_password</pre> without hashing, storing the password in clear text.	<pre>python
employee.password = new_password # insecure
employee.save()
</pre>

Fixes (excerpt)

```
# 1. Enforce authentication
permission_classes = [permissions.IsAuthenticated]

# 2. Issue JWT after OTP verification
token = RefreshToken.for_user(employee).access_token
return Response({'token': str(token)})

# 3. Use cryptographic randomness
import secrets
otp = secrets.randbelow(900000) + 100000

# 4. Check expiration
if timezone.now() > employee.otp_created_at + timedelta(minutes=5):
    return Response({'detail': 'OTP expired'}, status=400)

# 5. Hash password
```

```
employee.set_password(new_password)
employee.save()
```

2.

server/tracker/serializers.py

#	Vulnerability	Why it matters	Code snippet
1	Password not required on create	Allows creation of accounts with empty passwords, bypassing authentication.	python password = serializers.CharField(write_only=True, required=False) # vulnerable
2	Client can set password_hash	password_hash is write-only but not read-only; an attacker can supply a pre-computed hash, creating an account with a known password.	python validated_data.pop('password_hash', None) # but still accepted
3	Direct password_hash update	super().update() may persist a supplied password_hash, bypassing Django's hashing.	python user = super().update(instance=validated_data) # may write hash
4		While not a direct auth flaw, lack of permission	—

#	Vulnerability	Why it matters	Code snippet
	No permission checks in other serializers	enforcement can allow unauthorized CRUD on related objects.	

Fixes (excerpt)

```
# 1. Make password mandatory
password = serializers.CharField(write_only=True, required=True)

# 2. Remove write access to password_hash
class Meta:
    extra_kwargs = {'password_hash': {'write_only': True, 'read_only': True}}

# 3. Sanitize update
def update(self, instance, validated_data):
    validated_data.pop('password_hash', None)
    password = validated_data.pop('password', None)
    user = super().update(instance, validated_data)
    if password:
        user.set_password(password)
        user.save()
    return user
```

3.

server/config/settings.py

#	Vulnerability	Why it matters	Code snippet
1	DEBUG enabled in production	Exposes stack traces and secrets.	DEBUG = True

#	Vulnerability	Why it matters	Code snippet
2	Wildcard ALLOWED_HOSTS	Allows HTTP Host header attacks.	<pre>ALLOWED_HOSTS = ['*']</pre>
3	Hard-coded SECRET_KEY	Enables session cookie forgery.	<pre>SECRET_KEY = 'django-insecure-sola prod'</pre>
4	Hard-coded email password	Credential leakage.	<pre>EMAIL_HOST_PASSWORD = "whykkcbcxqk"</pre>
5	Hard-coded DB credentials	Database compromise.	<pre>DATABASES['default']['USER'] = 'po'</pre>
6	CORSALLOWALL_ORIGINS = True	Exposes authenticated endpoints to any origin.	<pre>CORS_ALLOW_ALL_ORIGINS = True</pre>
7	Click-jacking middleware commented out	Allows iframe phishing.	<pre># 'django.middleware.clickjacking.XF'</pre>

Fixes (excerpt)

```
DEBUG = os.getenv('DJANGO_DEBUG', 'False') == 'True'
ALLOWED_HOSTS = ['example.com', 'api.example.com']
SECRET_KEY = os.getenv('DJANGO_SECRET_KEY')
EMAIL_HOST_PASSWORD = os.getenv('EMAIL_HOST_PASSWORD')
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.postgresql',
        'NAME': os.getenv('POSTGRES_DB'),
```

```

        'USER': os.getenv('POSTGRES_USER'),
        'PASSWORD': os.getenv('POSTGRES_PASSWORD'),
        'HOST': os.getenv('POSTGRES_HOST', 'localhost'),
        'PORT': os.getenv('POSTGRES_PORT', '5432'),
    }
}
CORS_ALLOWED_ORIGINS = ['https://app.example.com', 'https://api.example.com']
MIDDLEWARE.append('django.middleware.clickjacking.XFrameOptionsMiddleware')

```

4.

server/reset_pass.py

#	Vulnerability	Why it matters	Code snippet
1	Hard-coded new password	Anyone who reads the script learns the password for all employees.	<pre>NEW_PASSWORD = "Password@1122334455"</pre>
2	Mass password reset without checks	Resets every employee to the same password, potentially locking out legitimate users.	<pre>python for emp in employees: emp.set_password(NEW_PASSWORD) emp.save()</pre>

Fixes (excerpt)

```
import os, getpass, re
from django.core.exceptions import ValidationError

def validate_password(pwd):
    if len(pwd) < 12 or not re.search(r'\d', pwd) or not re.search(r'[\^w\s]', pwd):
        raise ValidationError("Password does not meet complexity requirements.")

def reset_all_passwords():
    NEW_PASSWORD = os.getenv("NEW_EMPLOYEE_PASSWORD") or getpass.getpass("Enter new password")
    validate_password(NEW_PASSWORD)
    for emp in Employee.objects.all():
        emp.set_password(NEW_PASSWORD)
        emp.save()
```

5.

client/services/api.ts

#	Vulnerability	Why it matters	Code snippet
1	No Authorization header	All protected endpoints are called without a bearer token or session cookie.	ts const res = await fetch(`\${ {API_BASE}/employees/ `);
2	Missing credentials: 'include'	Browser will not send session cookies, breaking session-based auth.	ts fetch(url, { method: 'GET' });
3	No CSRF token for POST/PUT	If the backend uses CSRF protection (e.g., Django), requests will be	ts fetch(url, { method: 'POST', headers: {'Content-Type': 'application/json'} });

#	Vulnerability	Why it matters	Code snippet
		rejected or can be forged.	

Fixes (excerpt)

```
const authHeaders = (token?: string) => ({
  ...(token ? { Authorization: `Bearer ${token}` } : {}),
  'Content-Type': 'application/json',
});

const csrfToken = getCookie('csrftoken');

const getEmployees = async (token: string): Promise<Employee[]> => {
  const res = await fetch(`${API_BASE}/employees/`, {
    headers: authHeaders(token),
    credentials: 'include',
    headers: { ...authHeaders(token), 'X-CSRFToken': csrfToken },
  });
  if (!res.ok) throw new Error('Failed to fetch employees');
  return res.json();
};
```

6.

client/components/LoginScreen.tsx

&

client/components/ChangePasswordModal.tsx

No authentication-failure issues detected in these components.

Overall Assessment

The codebase contains multiple authentication-failure vectors:

1. **Unprotected API endpoints** and **missing token issuance** in the backend.
2. **Weak OTP generation** and **no expiration checks**.
3. **Plain-text password handling** and **hard-coded credentials**.
4. **Missing authentication headers** and **CSRF protection** on the client side.
5. **Hard-coded secrets** and **permissive settings** in

```
settings.py
```

These weaknesses collectively allow attackers to bypass authentication, compromise user accounts, and potentially gain full control of the system.

Recommendation:

Implement the fixes outlined above, enforce strict permission classes, use secure random generators, hash all passwords, externalize secrets, and ensure the client always sends authenticated requests with proper CSRF tokens. After remediation, re-run a full penetration test to confirm that all authentication failures have been eliminated.

Software or Data Integrity Failures – Overall Assessment

- Score: 7.5/10
 - Severity: High
-

1.

`server/seed.py`

– Data-Integrity via Missing Transaction

Issue – Bulk

```
delete()
```

calls are performed outside a transaction. A crash after a few deletes leaves the database partially cleared, breaking referential integrity.

Code Snippet

```
ChatMessage.objects.all().delete()  
AuditLog.objects.all().delete()  
Issue.objects.all().delete()  
...
```

Fix – Wrap the entire seeding logic in

```
transaction.atomic()
```

so the operation is all-or-nothing.

2.

`server/wipe_db.py`

– Destructive Script Without Safeguards

Issue – The script drops the public schema without environment checks, confirmation, or transaction safety. Running it in production would wipe the database and could leave it in a broken state if the

```
CREATE SCHEMA
```

fails.

Code Snippet

```
cursor.execute("DROP SCHEMA public CASCADE;")
cursor.execute("CREATE SCHEMA public;")
```

Fix – Add an environment guard, explicit confirmation prompt, wrap in

```
transaction.atomic()
```

, and catch exceptions.

3.

`server/reset_pass.py`

– Hard-coded Password & No Transaction

Issue – All employee accounts are reset to the same hard-coded password, with no confirmation or atomicity. A crash midway leaves some accounts updated and others not, corrupting authentication data.

Code Snippet

```
NEW_PASSWORD = "Password@1122334455"
for emp in employees:
```

```
emp.set_password(NEW_PASSWORD)
emp.save()
```

Fix – Source the new password from an environment variable, prompt for confirmation, and perform the update inside a transaction.

4.

`server/tracker/views.py`

– OTP & Data Exposure Flaws

#	Vulnerability	Impact
1	OTP expiration not enforced	Attacker can reset password after the intended window.
2	Insecure OTP generation (<code>random.randint</code>)	Predictable OTPs enable brute-force attacks.
3	OTP stored in plain text	Database compromise gives attackers immediate reset capability.
4	Email exposed in API responses	Personal data leakage, phishing risk.

Code Snippet

```
if not res.ok:
    const errorData = await res.json();
    throw errorData;    // raw error exposed
```

Fix – Use

secrets

for OTP, hash OTPs, enforce expiration, and remove email from responses.

5.

`server/tracker/models.py`

– Referential & Validation Gaps

#	Issue	Impact
1	<code>end_date</code> < <code>start_date</code> allowed	Invalid scheduling data.
2	<code>TestStep.sr_no</code> not unique per script	Ordering confusion.
3	Wrong default <code>IssueTeam.ibm</code>	Runtime error, data corruption.
4	<code>step_reference</code> free-form	Stale references.

Code Snippet

```
start_date = models.DateField()  
end_date = models.DateField(null=True, blank=True)
```

Fix – Add

```
clean()
```

validation,

```
unique_together
```

, correct enum default, and use FK for step reference.

6.

`server/tracker/serializers.py`

– Missing Cross-Field Validation

#	Issue	Impact
1	<pre>step</pre> may belong to another script	Referential integrity breach.
2	<pre>loggedByCode</pre> vs <pre>loggedBy</pre> mismatch	Audit trail confusion.
3	Chronological dates unchecked	Impossible timelines.
4	<pre>stepReference</pre> not validated	Mismatched references.
5	<pre>steps</pre> read-only	Incomplete script data.
6	Password optional	Inactive accounts.

Code Snippet

```
step = serializers.PrimaryKeyRelatedField(queryset=TestStep.objects.all(), required=False)
```

Fix – Implement

```
validate_step
```

```
,
```

```
validate
```

```
,
```

```
validate_stepReference
```

```
, make
```

```
steps
```

writable, and enforce password requirement.

7.

client/services/api.ts

– Unencrypted Traffic & No Response Integrity

#	Issue	Impact
1	Uses plain HTTP (http://)	MITM can tamper with requests/responses.
2	No response validation	Client trusts potentially corrupted data.
3	Raw error objects returned	Exposes internal stack traces.

Code Snippet


```
const SERVER_URL = 'http://siil-gw-drt-vm/uat-solar';
```

Fix – Force HTTPS, validate JSON against a schema or use signed tokens, and sanitize error handling.

Summary

The repository contains multiple **Software or Data Integrity Failures** ranging from missing transactional boundaries to insecure data handling and lack of validation. The highest severity is **High**, with an overall score of **7.5/10**. Implementing the suggested fixes will significantly reduce the risk of data corruption, unauthorized state changes, and exposure of sensitive information.

Security Vulnerability Report – Security Logging & Alerting Failures

Repository scanned:

/home/ubuntu/Documents/GAURAV's WORK/VAPT/cloned_code

Item	Description	Code Snippet	Impact	Fix
Vulnerability	Security Logging & Alerting Failures	—	—	—
Score	8/10	—	—	—
Severity	Critical	—	—	—

1.

server/config/settings.py

Issue	Lines	Why it matters	Fix
No explicit logging configuration	1-108	Django falls back to default console logger → security events (failed logins, CSRF, etc.) are not captured or alerted.	Add a <pre>LOGGING</pre> dictionary that logs <pre>django.security</pre> , <pre>django.request</pre> , and sends critical logs to a file and/or email.
			Set

Issue	Lines	Why it matters	Fix
<pre>DEBUG = True</pre> in production	11	Exposes stack traces and sensitive data.	<pre>DEBUG = False</pre> and configure <pre>ALLOWED_HOSTS</pre> .
<pre>ALLOWED_HOSTS = ['*']</pre>	13	Allows host-header injection.	Restrict to actual domain names.

Example addition

```
LOGGING = {
    'version': 1,
    'disable_existing_loggers': False,
    'formatters': {'verbose': {'format': '[{asctime}] {levelname} {name} {message}', 'style': '%'},
    'handlers': {
        'file': {'level': 'WARNING', 'class': 'logging.FileHandler',
                'filename': os.path.join(BASE_DIR, 'logs', 'security.log'),
                'formatter': 'verbose'},
        'mail_admins': {'level': 'ERROR',
                        'class': 'django.utils.log.AdminEmailHandler',
                        'include_html': True},
    },
    'loggers': {
        'django.security': {'handlers': ['file', 'mail_admins'],
                            'level': 'WARNING', 'propagate': False},
        'django.request': {'handlers': ['file', 'mail_admins'],
                            'level': 'ERROR', 'propagate': False},
    },
}
```

2.

`server/tracker/views.py`

Issue	Lines	Why it matters	Fix
No audit log for failed login	62-70	Brute-force attempts go unnoticed.	Log each failure with IP, user-code, timestamp.
No audit log for successful login	70-80	Successful logins are invisible.	Log each success.
<pre>print</pre> on email failure	70-75	No persistence or alert.	Replace with <pre>logger.error</pre> and create an <pre>AuditLog</pre> .
No audit log for OTP expiration / mismatch	120-135	Attackers can repeatedly request OTPs.	Log each expiration/mismatch.
No audit log for forgot-password initiation / confirmation	142-190	Sensitive actions lack traceability.	Log initiation, OTP send, reset confirmation, and failures.
No audit log for OTP clearing	140-145	Clearing OTP is a	Log it.

Issue	Lines	Why it matters	Fix
		security event.	
No audit logger used	Throughout	No security events are recorded.	Create security_logger = logging.getLogger('security') and use it.

Sample fix

```

security_logger = logging.getLogger('security')

@action(detail=False, methods=['post'])
def login(self, request):
    ...
    if not employee.check_password(password):
        security_logger.warning(
            "Failed login for code %s from IP %s", code, ip)
        return Response({"detail": "Invalid credentials."},
                        status=status.HTTP_401_UNAUTHORIZED)

    security_logger.info(
        "Successful login for code %s from IP %s", code, ip)
    ...

```

3.

server/tracker/admin.py

Issue	Lines	Why it matters	Fix
No audit trail for password changes	35-55	Password changes are silent.	Log in clean_new_password

Issue	Lines	Why it matters	Fix
			and save .
No audit trail for employee create/update/delete	90-110	Critical data changes are untracked.	Override save_model and delete_model to write to AuditLog .
No audit trail for script imports	140-160	Import failures are silent.	Log each import attempt and skipped rows.
No audit trail for password validation failures	45-55	Repeated validation failures are unlogged.	Log each failure.
AuditLog model unused	—	Intended audit mechanism never used.	Ensure all critical actions write to it.

Sample fix

```
class EmployeeAdmin(ImportExportModelAdmin):
    def save_model(self, request, obj, form, change):
        super().save_model(request, obj, form, change)
        action = 'Updated' if change else 'Created'
        AuditLog.objects.create(
            user=request.user,
            action=action,
```

```
        details=f'{action} employee {obj.code}'  
    )
```

4.

server/tracker/serializers.py

Issue	Lines	Why it matters	Fix
No audit logging for create/update	24-38	Critical data changes are untracked.	Log each create/update with <pre>logging.getLogger('audit')</pre> .
No audit logging for issue history access	70-78	Access to issue history is unlogged.	Log each access.

Sample fix

```
logger = logging.getLogger('audit')  
  
def create(self, validated_data):  
    user = Employee.objects.create(**validated_data)  
    logger.info(f"Employee created: id={user.id}, code={user.code}")  
    return user
```

5.

server/tracker/models.py

Issue	Lines	Why it matters	Fix
	71-133		Add

Issue	Lines	Why it matters	Fix
No audit trail for Script/TestStep/Issue changes		Business-critical objects are untracked.	post_save / post_delete signals that write to AuditLog .
Password changes not logged	64-68	Silent password changes.	Wrap set_password to log.
OTP usage not logged	58-59	OTP issuance/verification untracked.	Log OTP events.
No alert on failed login attempts	–	Brute-force attacks go unnoticed.	Log each failed attempt and trigger alert after threshold.
AuditLog unused	–	Intended audit mechanism never used.	Ensure all critical actions write to it.

Signal example

```
@receiver(post_save, sender=Script)
def script_saved(sender, instance, created, **kwargs):
    action = 'created' if created else 'updated'
    AuditLog.objects.create(
        script=instance,
        user_code=getattr(instance, '_audit_user', 'system'),
        action=action,
```



```
        details=str(instance.__dict__)
    )
```

6.

server/reset_pass.py

Issue	Lines	Why it matters	Fix
Hard-coded password logged in plain text	12-32	Exposes credentials.	Remove password from logs; use env variable.
<pre>print</pre> instead of logging	14-32	No persistent audit trail.	Use <pre>logging</pre> and avoid printing sensitive data.
No error handling	–	Silent failures.	Wrap in <pre>try/except</pre> and log exceptions.

Secure version

```
import os, logging, django
from django.core.exceptions import ImproperlyConfigured

os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'config.settings')
django.setup()

logger = logging.getLogger(__name__)
NEW_PASSWORD = os.getenv('EMPLOYEE_RESET_PASSWORD')
if not NEW_PASSWORD:
    logger.error("EMPLOYEE_RESET_PASSWORD not set")
    raise ImproperlyConfigured("Missing reset password")
```

```
# ... rest of script using logger instead of print ...
```

7.

server/wipe_db.py

Issue	Lines	Why it matters	Fix
<code>print</code> only	5	No persistent audit trail.	Use <code>logging</code> with file handler.
No user context	5	Cannot trace who ran the script.	Log <code>getpass.getuser()</code> or request user.
No alerting	5	Destructive action goes unnoticed.	Send email/Slack alert after wipe.

Secure version

```
import logging, getpass
from django.db import connection

logger = logging.getLogger('wipe_db')
user = getpass.getuser()
logger.info("⚠ WARNING: Deleting all data...", extra={'user': user})
with connection.cursor() as cursor:
    cursor.execute("DROP SCHEMA public CASCADE;")
    cursor.execute("CREATE SCHEMA public;")
logger.info("✅ Database wiped successfully.", extra={'user': user})
```

8.

server/seed.py

Issue	Lines	Why it matters	Fix
print only	1-4	No persistent audit trail.	Use logging .
No error handling	5-7	Silent failures.	Wrap in try/except and log.
Bulk delete without audit	10-12	Destructive operation untracked.	Log deletions.
Bulk create with hard-coded password	15-18	Security risk.	Log each creation; enforce password change.
No alert on failures	31-33	Silent failures.	Log and send email/ Slack.
No audit for scenario creation	38-70	Business logic changes untracked.	Log each creation.
No alert on issue creation	71-73	Failure to notify responsible team.	Log and trigger alert.

Secure version

```
import logging, django
from django.core.mail import send_mail

logger = logging.getLogger(__name__)
```

```

logger.setLevel(logging.INFO)
handler = logging.FileHandler('seed.log')
handler.setFormatter(logging.Formatter('%(asctime)s %(levelname)s %(message)s'))
logger.addHandler(handler)

def run_seed():
    try:
        logger.info("Cleaning database...")
        for model in [ChatMessage, AuditLog, Issue, TestStep, Script, Employee]:
            count = model.objects.all().delete()[0]
            logger.info(f"Deleted {count} records from {model.__name__}")

        # create users, set passwords, etc.
        # log each step
    except Exception as exc:
        logger.exception("Seed failed")
        send_mail("Seed script failure", str(exc), "noreply@example.com",
                  ["admin@example.com"])
        raise

```

9.

client/services/api.ts

Issue	Lines	Why it matters	Fix
No client-side logging of failed auth	12-45	Brute-force attempts unlogged.	Call audit endpoint on res.ok == false
Silent audit log failure	120-130	Audit failures hidden.	Throw error or reject promise.
No alert for critical ops	70-200	Failures may go unnoticed.	Wrap each fetch in try/catch, log to Sentry or audit endpoint.

Issue	Lines	Why it matters	Fix
No retry logic	12-200	Repeated failures can flood UI.	Implement exponential back-off.
Potential sensitive data in error messages	12-45	Stack traces may leak.	Sanitize error payload before throwing/logging.

Sample wrapper

```
const fetchWithAudit = async (url: string, opts: RequestInit, event?: string) => {
  try {
    const res = await fetch(url, opts);
    if (!res.ok) {
      const err = await res.json();
      if (event) await api.createLog({ event, details: err.detail });
      throw new Error(err.detail || 'Request failed');
    }
    return res;
  } catch (e) {
    if (event) await api.createLog({ event, details: e.message });
    throw e;
  }
};
```

Overall Assessment

- **Score:** 8/10
- **Severity: Critical** – The application lacks comprehensive audit logging, alerting, and error handling for all security-relevant operations.
- **Impact:** Attackers can perform credential stuffing, OTP abuse, or destructive actions without detection. Administrators cannot detect or investigate incidents.
- **Recommendation:** Implement the fixes above, enforce a central audit logger, and integrate with an external monitoring/alerting system (e.g., Sentry, Slack, PagerDuty). Ensure all destructive or sensitive operations are logged with user context, timestamps, and persisted to a tamper-resistant store.

Vulnerability Report – Mishandling of Exceptional Conditions**Score:** 8/10**Severity:** Critical**1.****server/tracker/views.py**

Issue	Why it's a problem	Code snippet
Generic exception swallowing in _send_otp	Catches <i>any</i> exception, swallows it, and only prints to stdout. The OTP is already persisted before the email is sent, so a failed email leaves a usable OTP in the DB.	<pre>python
try:
send_mail(...)
 return True
except Exception as e:
 print(f"Email Error: {e}")
 return False
</pre>
Generic exception swallowing in login	Same as above – no proper logging, OTP already persisted.	<pre>python
try:
send_mail(...)
except Exception as e:
 print(f"Email Error: {e}")
 return Response(...)
</pre>
Potential TypeError in	If otp_created_at is	<pre>python
if timezone.now() > employee.otp_created_at + timedelta(minutes=5):
 return Response(...)
</pre>

Issue	Why it's a problem	Code snippet
verify_otp	None , adding a timedelta raises a TypeError , causing a 500 error.	
OTP left in DB on email failure in forgot_password_init	_send_otp returns False but the OTP remains stored, allowing replay attacks.	<pre>python if self._send_otp(employee): return Response(...) else: return Response(...)</pre>

Fix – Use

```
transaction.atomic()
```

, catch specific exceptions, log properly, and roll back the OTP write on failure.

2.

```
server/tracker/models.py
```

Issue	Why it's a problem	Code snippet
set_password accepts any input	make_password() expects a non-empty string; passing	<pre>python def set_password(self, raw_password):</pre>

Issue	Why it's a problem	Code snippet
	None or non-string raises TypeError . No validation or exception handling.	self.password_hash = make_password(raw_password)

check_password accepts any input	check_password() also expects a string; passing None raises TypeError . No handling.	python
def check_password(self, raw_password):
 return check_password(raw_password, self.password_hash)

Fix – Validate input, catch exceptions, and return a clear error or

False

3.

server/tracker/serializers.py

Issue	Why it's a problem	Code snippet
Broad except Exception in	Swallows all exceptions, hides bugs, and	python
try:
 script_logs = obj.script.history.all()
 return AuditLogSerializer(...).data
except Exception:
 return []

Issue	Why it's a problem	Code snippet
<code>get_history</code>	silently returns an empty list.	

Fix – Log the exception, catch only expected errors, and surface unexpected ones.

4.

`server/reset_pass.py`

Issue	Why it's a problem	Code snippet
No error handling during bulk password reset	Any DB error aborts the script, leaving the database partially updated.	<pre>python for emp in employees: emp.set_password(NEW_PASSWORD) emp.save()</pre>

Fix – Wrap the operation in

```
transaction.atomic()
```

, catch

```
DatabaseError
```

/

```
IntegrityError
```

, and log failures.

5.

server/seed.py

Issue	Why it's a problem	Code snippet
Destructive DB ops without transaction or error handling	A failure during DROP SCHEMA or CREATE SCHEMA leaves the DB in an inconsistent state.	python ChatMessage.objects.all()
bulk_create and subsequent save() calls lack exception handling	Any integrity violation aborts the script, leaving partial data.	python created_users = Employee.objects.create(created_users: user.set_password('1234567890'))
Script.objects.create , TestStep.objects.create , Issue.objects.create	Each can raise IntegrityError ; no handling.	python script = Script.objects.create(...) TestStep.objects.create(...)

Fix – Use

transaction.atomic()

for cleanup, wrap each create in

try/except

, log errors, and skip or rollback problematic records.

6.

server/wipe_db.py

Issue	Why it's a problem	Code snippet
Destructive schema drop/create without transaction or error handling	If the second statement fails, the schema remains dropped, leaving the DB unusable.	<pre>python
cursor.execute("DROP SCHEMA public CASCADE;")
cursor.execute("CREATE SCHEMA public;")
</pre>

Fix – Wrap both statements in

```
transaction.atomic()
```

and add a

```
try/except
```

block to log failures.

7.

client/services/pdfGenerator.ts

Issue	Why it's a problem	Code snippet
<pre>scenario.name</pre> may be undefined	<pre>splitTextToSize</pre> expects a string; <pre>undefined</pre>	<pre>ts
const scenarioTitle = doc.splitTextToSize(scenario.name, 180);
</pre>

Issue	Why it's a problem	Code snippet
	causes a runtime error.	
scenario.steps may be undefined/not an array	.map / .forEach will throw.	<pre>ts
const responseBody = scenario.steps.map(...);
</pre>
doc.lastAutoTable may be undefined	Accessing .finally throws.	<pre>ts
let finally = doc.lastAutoTable.finally + 15;
</pre>
scenario.ibmResponsible / solarResponsible may be undefined	.toUpperCase() on undefined throws.	<pre>ts
addText(`NAME: \$ {sigNamesRow1[i].toUpperCase()}` , ...);
</pre>

Fix – Add guards and default values (

??

,

Array.isArray

, optional chaining) before accessing properties.

Overall Assessment

The codebase contains multiple instances where exceptions are swallowed, resources are not cleaned up, or critical operations are performed without transactional safety. These patterns expose the application to data corruption, silent failures, and potential security risks such as replayable OTPs and exposed stack traces.

Recommendation

- Adopt consistent error-handling patterns: use specific exception types, log errors, and fail loudly when unexpected conditions arise.
- Wrap destructive or multi-step operations in database transactions (

```
transaction.atomic()
```

in Django) to guarantee atomicity.

- Validate inputs before processing and provide safe defaults for optional data in client-side code.

Implementing these changes will elevate the application's resilience and reduce the risk of mishandling exceptional conditions.

